



Sprint 1: Business Understanding y Data Understanding

Product Backlog, Sprint Planning y Roles del
Equipo

Pedro Shiguihara, pshiguihara@pucp.edu.pe

27 de marzo de 2026

Agenda

1. Contexto: del prototipo al MVP en un proyecto de clasificación
2. Metodología: CRISP-DM y Scrum aplicados
3. Product Backlog del proyecto
4. Sprint 1: Alcance y Sprint Planning
5. Roles del equipo (5 personas)
6. Detalle de tareas por rol
7. Entregables del Sprint 1
8. Criterios de aceptación y Definition of Done

Contexto del Proyecto

Un proyecto de clasificación con datasets reales

🎯 Objetivo General

Cada equipo de 5 personas abordará un **problema de clasificación** usando uno de los datasets de la cartera de proyectos. El objetivo es construir un **MVP** iterativo usando **Jupyter Notebooks**, aplicando CRISP-DM dentro de un marco ágil (Scrum).

🗄️ Datasets Disponibles

- Cancelaciones Hoteleras (119K reg., 32 var.)
- Conversión E-Commerce (12K reg., 18 var.)
- Telemarketing Bancario (41K reg., 21 var.)
- Riesgo Crediticio (30K reg., 25 var.)
- Segmentación por Ingreso (48K reg., 15 var.)
- Y 5 proyectos más de la cartera

🔧 Herramientas Base

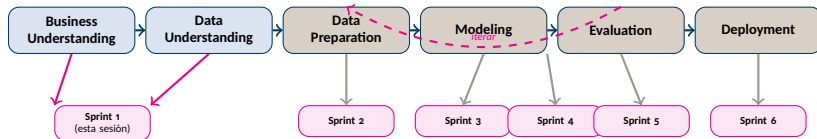
- 🐍 Python 3.10+
- 📓 Jupyter Notebooks
- 🔗 Git + GitHub
- 📋 GitHub Projects (Kanban)
- 📦 Conda (entornos)
- 🗄️ DVC (datos)
- ⚙️ ipywidgets (prototipos)

⚠️ Clave

Todos los proyectos son de **clasificación binaria**: predecir si un evento ocurre o no.

Metodología: CRISP-DM dentro de Scrum

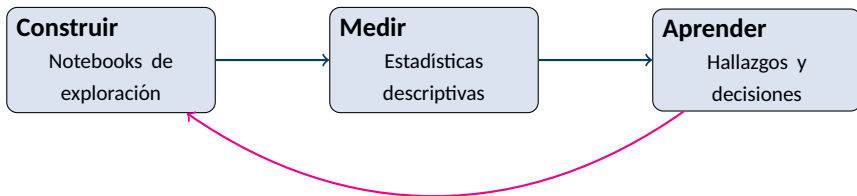
Mapeo de fases CRISP-DM a 6 Sprints



- **Sprint 1:** Business & Data Understanding.
- **Sprint 2:** Data Preparation.
- **Sprint 3:** Modeling (baseline).
- **Sprint 4:** Modeling avanzado + tuning.
- **Sprint 5:** Evaluation + Business Value.
- **Sprint 6:** Deployment en AWS (MVP).

Ciclo Lean Startup aplicado al Sprint 1

Construir → Medir → Aprender



💡 Filosofía del Sprint 1

No se construye ningún modelo aún. El MVP de este sprint es un conjunto de **Jupyter Notebooks documentados** que demuestran comprensión del negocio y de los datos, con visualizaciones interactivas (ipywidgets) que permitan a un stakeholder explorar los datos.

Product Backlog Completo

Historias de usuario para todo el proyecto (6 sprints)

ID	Historia de Usuario	Fase CRISP-DM	Sprint
PB-01	Definir el problema de negocio y objetivos	Business Underst.	1
PB-02	Identificar criterios de éxito del modelo	Business Underst.	1
PB-03	Cargar y explorar el dataset inicial	Data Understanding	1
PB-04	Analizar calidad de datos (nulos, tipos, duplicados)	Data Understanding	1
PB-05	Generar estadísticas descriptivas y visualizaciones	Data Understanding	1
PB-06	Construir prototipo interactivo de exploración	Data Understanding	1
PB-07	Limpiar y transformar variables	Data Preparation	2
PB-08	Realizar feature engineering	Data Preparation	2
PB-09	Balancear clases si es necesario	Data Preparation	2
PB-10	Entrenar modelos baseline de clasificación	Modeling	3
PB-11	Evaluar modelos baseline con métricas	Modeling	3
PB-12	Comparar modelos y seleccionar candidatos	Modeling	3
PB-13	Optimizar hiperparámetros del mejor modelo	Modeling	4
PB-14	Aplicar técnicas avanzadas (ensambles, stacking)	Modeling	4
PB-15	Validar modelo final (cross-validation, test set)	Modeling	4
PB-16	Evaluar Business Value del modelo seleccionado	Evaluation	5
PB-17	Construir dashboard interactivo de resultados	Evaluation	5
PB-18	Documentar hallazgos y recomendaciones de negocio	Evaluation	5
PB-19	Empaquetar modelo como API (Flask/FastAPI)	Deployment	6
PB-20	Desplegar prototipo MVP en AWS (EC2/Lambda)	Deployment	6
PB-21	Integrar modelo con dashboard desplegado	Deployment	6

→ Las filas resaltadas en azul corresponden al **Sprint 1** (esta iteración).

Sprint 1: Sprint Planning

Alcance: Business Understanding + Data Understanding

Meta del Sprint

Lograr una **comprensión integral** del problema de negocio y del dataset asignado, documentada en Jupyter Notebooks reproducibles con exploración interactiva.

Duración

- **1 semana** de trabajo colaborativo
- Daily stand-ups (async vía GitHub Issues)
- Sprint Review al final de la semana

Sprint Backlog (6 ítems)

- PB-01** Definición del problema de negocio
- PB-02** Criterios de éxito del modelo
- PB-03** Carga y exploración inicial
- PB-04** Análisis de calidad de datos
- PB-05** Estadísticas y visualizaciones
- PB-06** Prototipo interactivo (ipywidgets)

Gestión

- Repositorio GitHub con GitHub Projects
- Cada tarea = 1 Issue en el board Kanban
- Ramas por feature: `feat/PB-XX`

Roles del Equipo: 5 Personas

Cada miembro asume un rol con responsabilidades claras para el Sprint 1



Project Manager
Planifica, coordina y gestiona el board Kanban en GitHub



Business Analyst
Define el problema de negocio, KPIs y criterios de éxito



Data Engineer
Carga datos, configura entorno Conda, DVC y estructura del repo



Data Analyst
Explora datos, calidad, estadísticas descriptivas y visualizaciones



Prototype Dev
Construye el notebook interactivo con ipywidgets

Nota importante

Los roles no son silos: todos colaboran y revisan el trabajo de los demás vía **Pull Requests** en GitHub. El rol define la **responsabilidad principal**, no exclusividad.

Rol 1: Project Manager

 Coordinación, planificación y seguimiento

Tareas del Sprint 1

1. Crear el repositorio en GitHub e invitar al equipo.
2. Configurar GitHub Projects con board Kanban (To Do, In Progress, In Review, Done).
3. Crear los Issues del Sprint 1 (PB-O1 a PB-O6) con descripción, assignee y labels.
4. Definir la estructura de carpetas del repositorio.
5. Realizar seguimiento diario del board y desbloquear impedimentos.
6. Preparar el Sprint Review (presentación de resultados).

Estructura del Repo

```
proyecto/  
+-- data/  
|   +-- raw/  
|   +-- processed/  
+-- notebooks/  
|   +-- 01_business.ipynb  
|   +-- 02_data_loading.ipynb  
|   +-- 03_eda.ipynb  
|   +-- 04_prototype.ipynb  
+-- environment.yml  
+-- .gitignore  
+-- .dvc/  
+-- README.md
```

Rol 2: Business Analyst

 Comprensión del negocio y definición de objetivos

Tareas del Sprint 1 (PB-O1 y PB-O2)

1. Investigar el contexto del sector/industria del dataset asignado.
2. Redactar el **objetivo de negocio** en el notebook `01_business.ipynb`.
3. Definir las **preguntas de negocio** que el modelo debe responder.
4. Establecer los **criterios de éxito**:
 - Métrica principal (accuracy, F1, AUC-ROC).
 - Umbral mínimo aceptable.
 - Restricciones del negocio.
5. Identificar **stakeholders** y sus necesidades.
6. Documentar supuestos y riesgos del proyecto.

Contenido del Notebook

El notebook `01_business.ipynb` debe incluir:

- Título y descripción del proyecto
- Contexto de la industria
- Problema a resolver (en lenguaje de negocio)
- Variable objetivo (*target*)
- KPIs de éxito del modelo
- Glosario de términos del dominio
- Referencias bibliográficas



Usar la información de `cartera_proyectos` como punto de partida, pero profundizar con investigación propia.

Rol 3: Data Engineer

 Infraestructura de datos y entorno reproducible

Tareas del Sprint 1 (PB-O3 parcial)

1. Crear el entorno Conda con `environment.yml`:
 - Python 3.10+, pandas, numpy, matplotlib, seaborn, scikit-learn, ipywidgets, jupyterlab.
2. Configurar **DVC** para versionar el dataset:
 - `dvc init`, `dvc add data/raw/dataset.csv`
 - Configurar remote storage.
3. Configurar el `.gitignore` adecuado.
4. Escribir el notebook `02_data_loading.ipynb`:
 - Carga del dataset con pandas.
 - Verificación de dimensiones, tipos y primeras filas.
5. Documentar instrucciones de setup en `README.md`.

Comandos Clave

```
# Crear entorno
conda create -n proyecto python=3.11
conda activate proyecto
# Instalar dependencias
conda install -c conda-forge \
    pandas numpy matplotlib \
    seaborn scikit-learn \
    ipywidgets jupyterlab
# Exportar entorno
conda env export --from-history \
    > environment.yml
# Inicializar DVC
dvc init
dvc add data/raw/dataset.csv
git add data/raw/.gitignore \
    data/raw/dataset.csv.dvc
```

Rol 4: Data Analyst

 Exploración, calidad y análisis descriptivo

Tareas del Sprint 1 (PB-O4 y PB-O5)

1. **Calidad** (03_eda.ipynb): nulos, tipos, duplicados, outliers.
2. **Descriptivas**: `describe()`, `value_counts()`, balance de clases.
3. **Visualizaciones**: histogramas, barras, heatmap de correlación, target vs. variables.

Preguntas Guía

Al explorar los datos, responder:

- ¿Cuántos registros y variables tiene el dataset?
- ¿Qué porcentaje de datos falta por columna?
- ¿Están balanceadas las clases del target?
- ¿Qué variables correlacionan con el target?
- ¿Hay variables redundantes o constantes?
- ¿Existen patrones temporales?

Librerías

- `matplotlib + seaborn`
- `pandas profiling`
- `plotly` (opcional)

Rol 5: Prototype Developer

 Notebook interactivo con ipywidgets

Tareas del Sprint 1 (PB-O6)

1. Construir `04_prototype.ipynb` con controles interactivos usando **ipywidgets**:
 - **Dropdown** para seleccionar variables a visualizar.
 - **Slider** para filtrar rangos numéricos.
 - **Checkbox** para activar/desactivar filtros.
 - **RadioButtons** para elegir tipo de gráfico.
2. Integrar las visualizaciones del Data Analyst con controles interactivos.
3. Permitir que un stakeholder no técnico explore los datos sin escribir código.
4. Documentar cada celda del notebook con explicaciones claras.

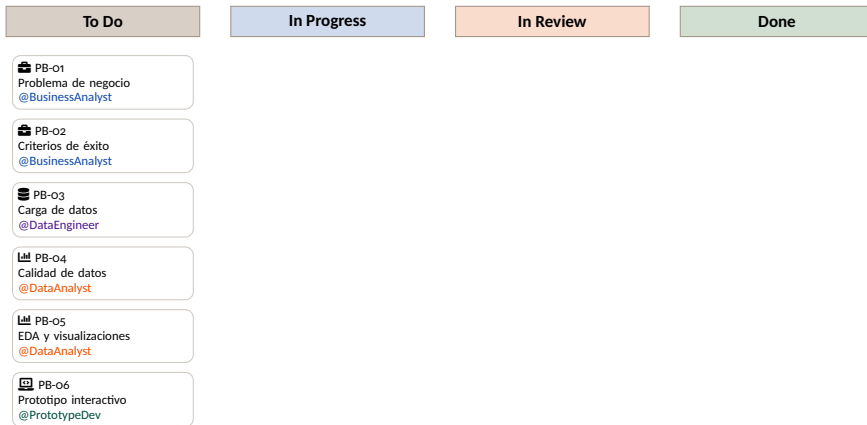
Ejemplo con interact

```
from ipywidgets import interact
import matplotlib.pyplot as plt
import seaborn as sns

@interact(
    variable=df.select_dtypes(
        'number').columns.tolist(),
    bins=(5, 50, 5)
)
def explorar(variable, bins):
    fig, ax = plt.subplots(1, 2,
                           figsize=(10, 4))
    # Histograma
    sns.histplot(df[variable],
                 bins=bins, ax=ax[0])
    # Boxplot por target
    sns.boxplot(x='target',
                y=variable,
                data=df, ax=ax[1])
    plt.tight_layout()
    plt.show()
```

Tablero Kanban en GitHub Projects

Visualización del flujo de trabajo del Sprint 1



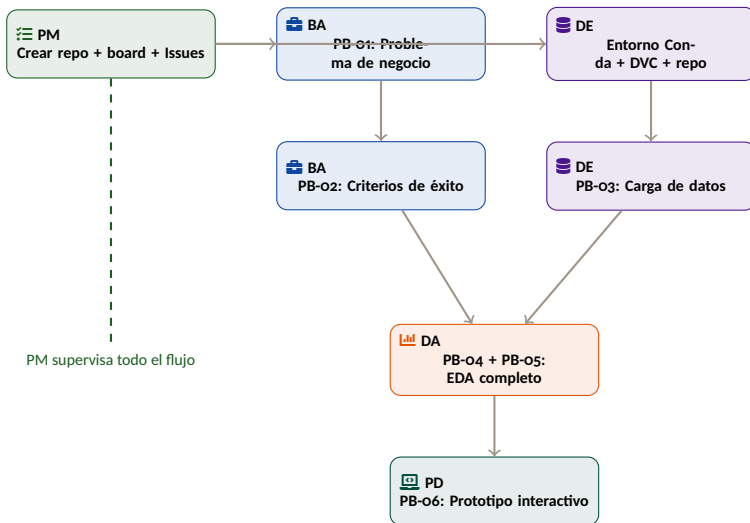
Flujo de Trabajo en el Kanban

Flujo

Cada Issue se mueve: **To Do** → se crea rama `feat/PB-XX` → **In Progress** → se abre Pull Request → **In Review** → se aprueba y mergea → **Done**.

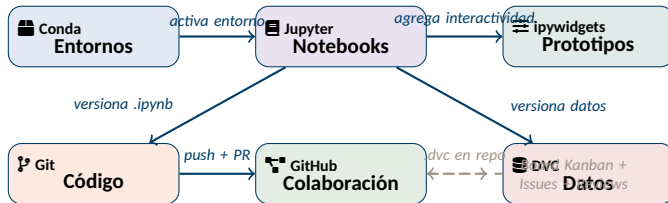
Dependencias entre Tareas

Orden sugerido de ejecución en el Sprint 1



Herramientas Base: ¿Cómo encajan juntas?

Visión integrada para principiantes



💡 Idea clave

Cada herramienta resuelve **un problema específico**. Juntas forman un flujo donde el código, los datos y la colaboración están versionados y organizados.

Tip 1: Flujo diario de trabajo

 Los pasos que cada miembro repite en cada sesión de trabajo

Al iniciar el día

```
# 1. Actualizar el repo
git pull origin main

# 2. Activar el entorno
conda activate proyecto

# 3. Traer datos actualizados
dvc pull

# 4. Crear rama para tu tarea
git checkout -b feat/PB-04

# 5. Abrir JupyterLab
jupyter lab
```

Al terminar el día

```
# 1. Guardar notebook
# (Ctrl+S en Jupyter)

# 2. Agregar cambios
git add notebooks/03_eda.ipynb

# 3. Commit descriptivo
git commit -m "feat(PB-04):
    agrega analisis de nulos
    y distribucion del target"

# 4. Push a GitHub
git push origin feat/PB-04

# 5. Abrir Pull Request en GitHub
# (si la tarea esta lista)
```

Recuerda

Nunca trabajes directamente en la rama main. Siempre usa ramas feat/PB-XX y Pull Requests.

Tip 2: Conda + DVC — Reproducibilidad

🔄 Que cualquier miembro pueda replicar tu trabajo exactamente



Conda: entorno compartido

- Todos usan el **mismo** `environment.yml`.
- Si instalas un paquete nuevo:

```
conda install -c conda-forge plotly
conda env export --from-history \
  > environment.yml
git add environment.yml
git commit -m "chore: agrega plotly"
```

- Los demás actualizan con:

```
conda env update -f environment.yml
```



DVC: datos fuera de Git

- Los CSV/archivos grandes **no** van a Git.
- DVC los versiona aparte:

```
# Al agregar un dataset nuevo
dvc add data/raw/dataset.csv
git add data/raw/dataset.csv.dvc
git add data/raw/.gitignore
git commit -m "data: agrega dataset"
git push && dvc push
```

- Los demás obtienen los datos con:

```
git pull && dvc pull
```

Tip 3: GitHub Issues + Projects — Organización

☰ Cómo conectar el trabajo diario con el tablero Kanban



Crear buenos Issues

Cada Issue del Sprint debe tener:

1. **Título claro:** "PB-04: Análisis de calidad de datos"
2. **Descripción:** qué se espera como resultado.
3. **Assignee:** quién es responsable.
4. **Labels:** sprint-1, data-understanding.
5. **Criterio de aceptación:** cómo saber que está listo.



Conectar commits con Issues

Al hacer commit, referencia el Issue:

- `git commit -m "feat(PB-04): ..."`
- En el PR, escribe Closes #4 para que el Issue se cierre automáticamente al mergear.

Flujo completo:

1. Mover Issue a **In Progress**.
2. Trabajar en rama feat/PB-04.
3. Abrir PR → mover a **In Review**.
4. Compañero revisa y aprueba.
5. Mergear → Issue pasa a **Done**.

Tip 4: Jupyter + ipywidgets — Del análisis al prototipo

 Cómo evolucionar un notebook estático a uno interactivo

Paso 1: Notebook estático

Primero el Data Analyst crea el EDA clásico:

- Carga de datos con `pandas`.
- Estadísticas con `df.describe()`.
- Gráficos con `matplotlib/seaborn`.
- Hallazgos escritos en celdas Markdown.

Este notebook (`03_eda.ipynb`) es el **análisis base** que documenta los descubrimientos.

Paso 2: Notebook interactivo

El Prototype Dev toma las visualizaciones clave y les agrega controles:

- **Dropdown:** elegir variable a graficar.
- **Slider:** ajustar número de bins.
- **Checkbox:** filtrar por categoría.
- **RadioButtons:** tipo de gráfico.

El resultado (`04_prototype.ipynb`) permite que un **stakeholder** explore los datos sin escribir código.



Tip

El Prototype Dev debe importar funciones del notebook EDA en lugar de duplicar código. Reutilizar > reescribir.

Errores comunes al comenzar

! Qué evitar y cómo resolverlo

	Error común	Solución
⊘	Subir el CSV directamente a GitHub	Usar DVC: <code>dvc add data/raw/archivo.csv</code>
⊘	Trabajar todos en la rama <code>main</code>	Cada tarea en su rama: <code>feat/PB-XX</code>
⊘	Instalar paquetes sin actualizar <code>environment.yml</code>	Siempre exportar: <code>conda env export --from-history</code>
⊘	Hacer notebooks sin celdas Markdown	Documentar cada sección: qué se hace y qué se encontró
⊘	Mergear sin revisión de código	Siempre usar Pull Requests con al menos 1 reviewer
⊘	No mover los Issues en el board	Actualizar el Kanban al iniciar y terminar cada tarea

Entregables del Sprint 1

Lo que cada rol debe entregar al final de la semana

Rol	Entregable	Formato
PM	Repositorio GitHub configurado con board Kanban, Issues creados y asignados, README con instrucciones de setup	GitHub Repo
BA	Notebook documentado con problema de negocio, contexto del sector, variable objetivo, KPIs y criterios de éxito	01_business.ipynb
DE	Entorno Conda exportado, DVC configurado, notebook de carga y verificación de datos	environment.yml + 02_data_loading.ipynb
DA	Notebook con análisis de calidad, estadísticas descriptivas, visualizaciones y hallazgos clave	03_eda.ipynb
PD	Notebook con prototipo interactivo usando ipywidgets que permita explorar los datos dinámicamente	04_prototype.ipynb

Definition of Done (DoD)

Criterios que cada tarea debe cumplir para considerarse terminada

✓ Criterios Generales

- ☐ Código en rama `feat/PB-XX` con Pull Request aprobado y mergeado a `main`.
- ☐ Notebook ejecutable de principio a fin sin errores (*Restart & Run All*).
- ☐ Celdas de markdown documentando cada sección del análisis.
- ☐ El entorno se reproduce con `conda env create -f environment.yml`.
- ☐ Issue correspondiente movido a "Done" en el board.

★ Criterios de Calidad

- ☐ Variables y funciones con nombres descriptivos.
- ☐ Visualizaciones con títulos, ejes etiquetados y leyendas.
- ☐ Hallazgos documentados en celdas Markdown (no solo código).
- ☐ Datos versionados con DVC (no subir CSV a Git).
- ☐ Al menos 1 revisión de código (PR review) por otro miembro del equipo.

Visión General: 6 Sprints hasta el Despliegue

Del entendimiento del negocio al MVP desplegado en AWS

Sprint	Fase CRISP-DM	Objetivo principal
1	Business + Data Underst.	Comprender el problema y explorar los datos
2	Data Preparation	Limpiar, transformar y crear features
3	Modeling	Entrenar modelos baseline y comparar
4	Modeling	Optimizar hiperparámetros y modelos avanzados
5	Evaluation	Evaluar Business Value y documentar resultados
6	Deployment	Desplegar el mejor modelo como MVP en AWS

Meta final (Sprint 6)

Un **prototipo funcional desplegado en AWS** (EC2, Lambda o SageMaker) que expone el mejor modelo de clasificación como API y/o dashboard interactivo, cumpliendo con el **Business Value** definido en el Sprint 1.

Resumen del Sprint 1

¿Qué?

Business Understanding
+ Data Understanding

¿Quién?

5 roles: PM, BA, DE, DA, PD

¿Cómo?

Jupyter Notebooks +
Git + GitHub Projects



Entregable

4 Notebooks documentados + repo configurado



Siguiente

Sprint 2: Data Preparation (limpieza + features)

! Recordar

- El Sprint 1 **no** incluye modelado. Se trata de **entender** antes de construir.
- La calidad del EDA en este sprint determina la calidad del modelo en los 5 sprints siguientes.
- El Business Value definido aquí guiará la selección del modelo final y su despliegue en AWS.

Checklist de Inicio

Pasos inmediatos para comenzar el Sprint 1

1. **PM:** Crear repositorio en GitHub e invitar a los 4 miembros.
2. **PM:** Configurar GitHub Projects con las columnas del Kanban.
3. **PM:** Crear los 6 Issues (PB-01 a PB-06) y asignarlos.
4. **DE:** Clonar el repo, crear `environment.yml` y hacer push.
5. **Todos:** Clonar el repo y crear el entorno con `conda env create -f environment.yml`.
6. **BA:** Iniciar investigación del sector y redactar `01_business.ipynb` en rama `feat/PB-01`.
7. **DE:** Descargar el dataset, configurar DVC y crear `02_data_loading.ipynb` en rama `feat/PB-03`.
8. **DA:** Una vez mergeado PB-03, iniciar EDA en `03_eda.ipynb` en rama `feat/PB-04`.
9. **PD:** Una vez avanzado el EDA, construir `04_prototype.ipynb` en rama `feat/PB-06`.
10. **Todos:** Sprint Review — presentar hallazgos y demostrar el prototipo.



Diplomado de Especialización en **Data Analytics**